OTUS

Kotlin Backend Developer.Professional

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Реализация алгоритмов ML в Kotlin-проекте

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

Маршрут вебинара

1. Задачи бэкенда для МО
2. Инфраструктура МО
3. Интеграции МО в проект
4. Применение МО в Kotlin
5. Рефлексия

Цели вебинара

После занятия вы сможете

1. Иметь представление о задачах, решаемых с помощью Искусственного интеллекта и Машинного обучения и способах их решения
2. Знать как МО интегрируется в ИТ-инфраструктуру и как происходит взаимодействие с другими компонентами системы
3. Решать задачи интеграции МО в приложение на Kotlin

Смысл

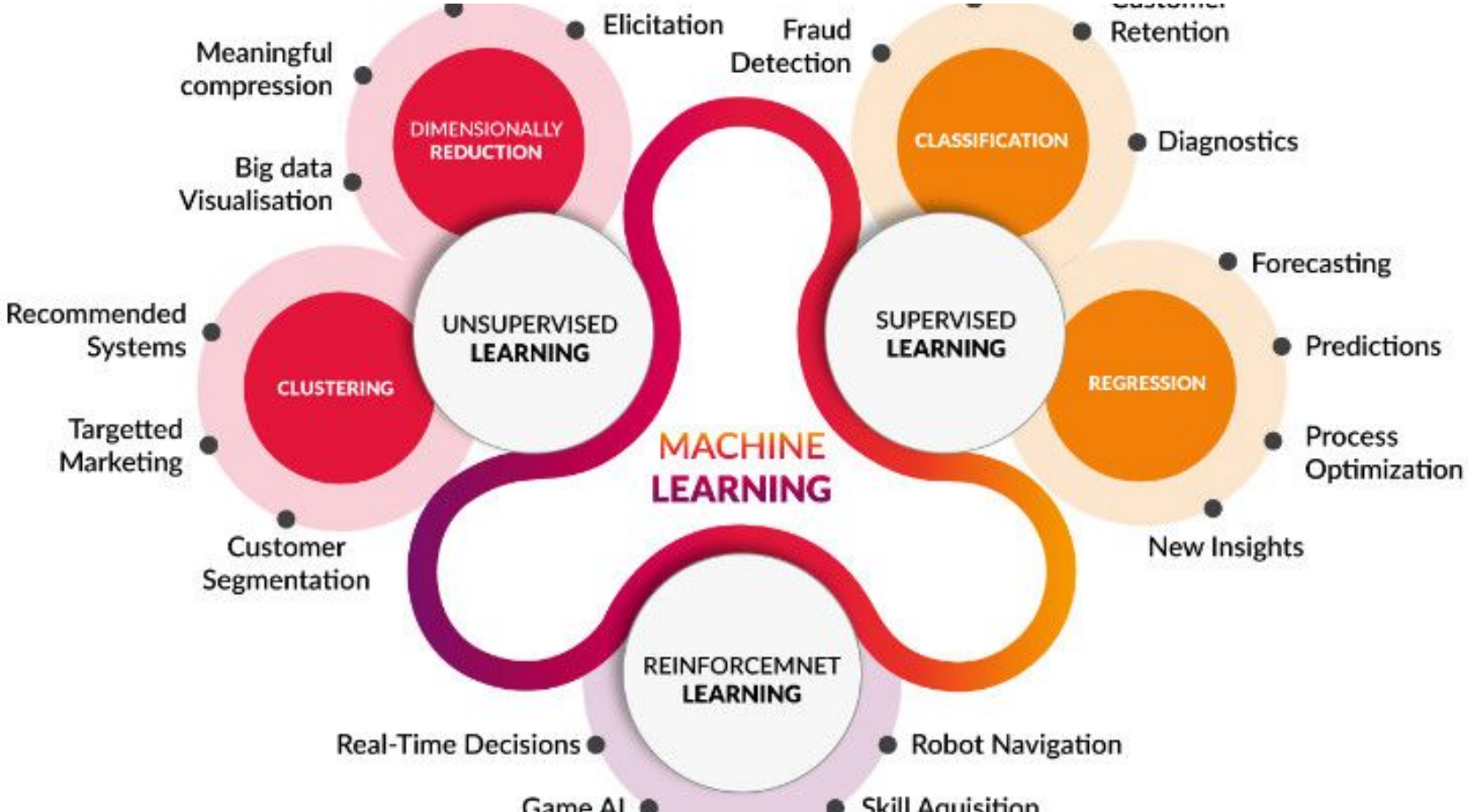
Зачем вам это уметь

1. Машинное обучение - это технологическая революция, оно прочно обосновалось на рынке и влияет на все сферы нашей жизни
2. Все чаще прилетают задачи на внедрение МО в системы
3. Kotlin вполне может использоваться для решения задач МО

Задачи для МО в бэкенде

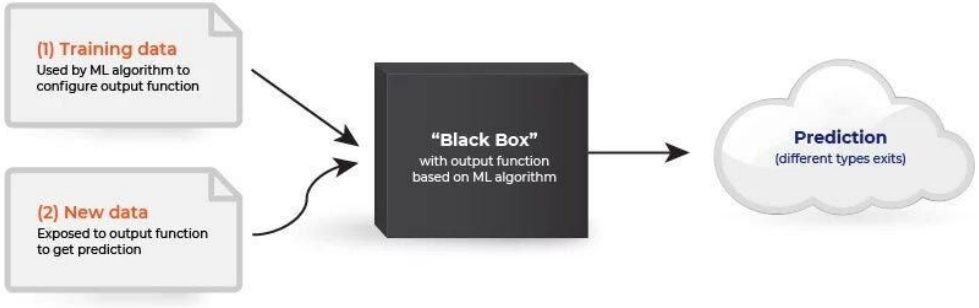
Задачи для МО

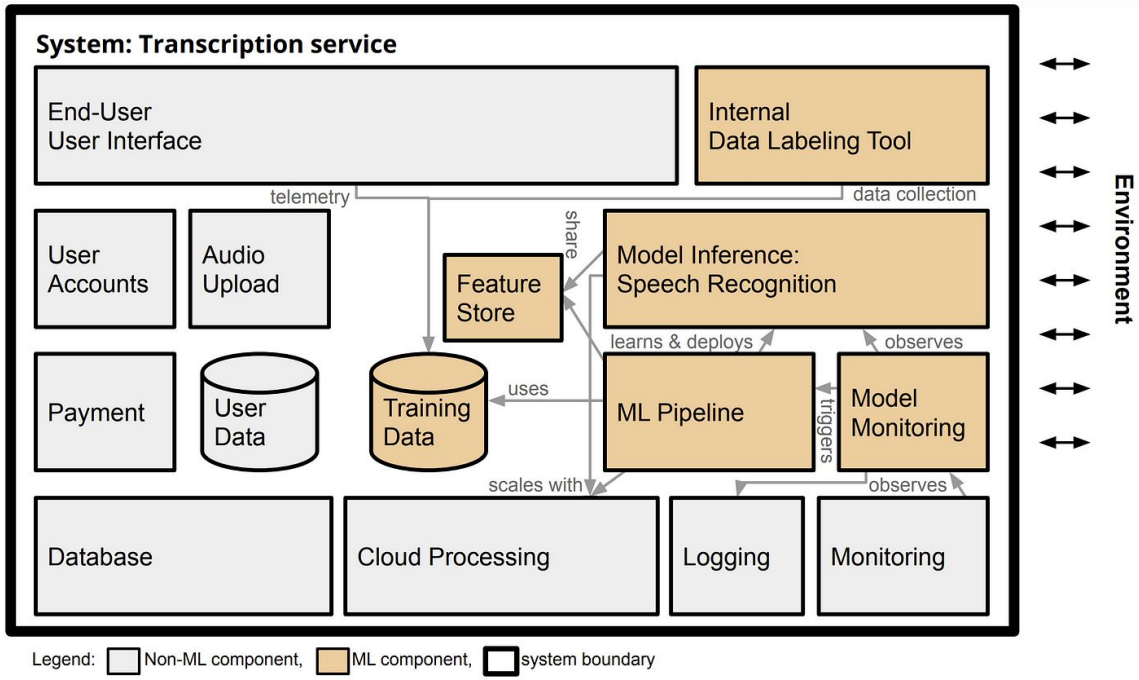
Общая карта задач для МО



Особенности МО в бэкенд-проектах

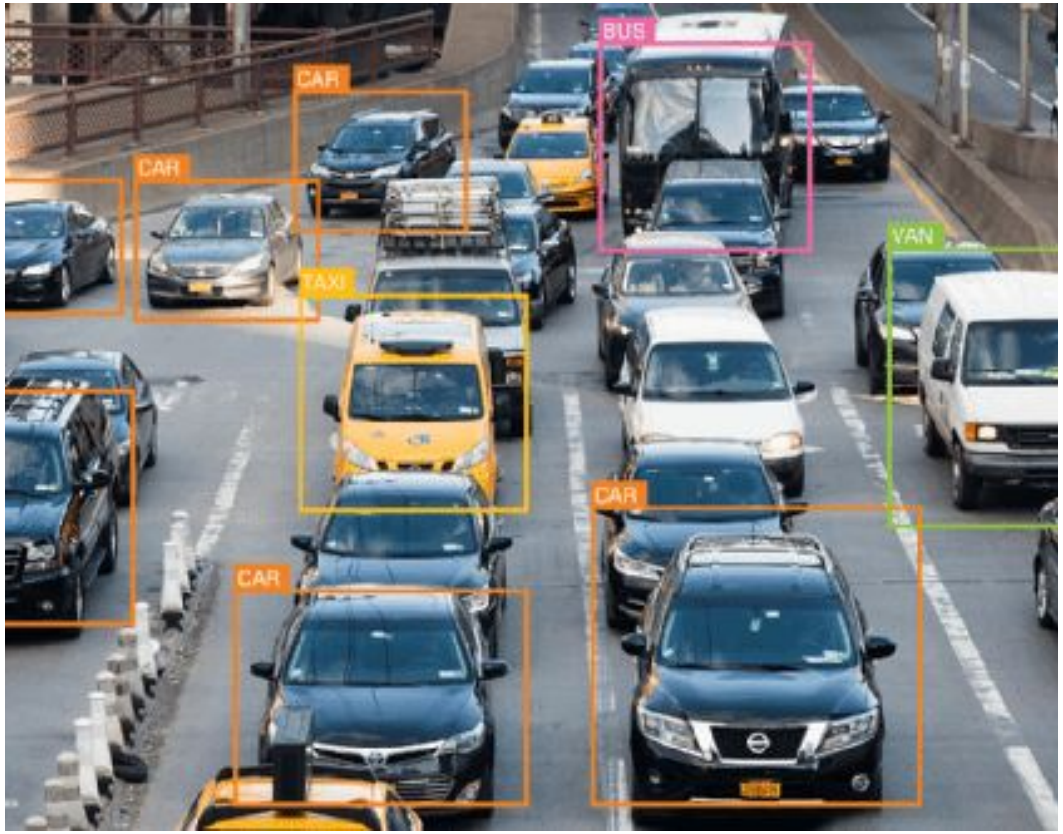
1) Очень ресурсоемкие
2) Универсальные решения дорогие
3) Набор специализированные решений
4) На текущий момент не заменяют бэкенд-разработку





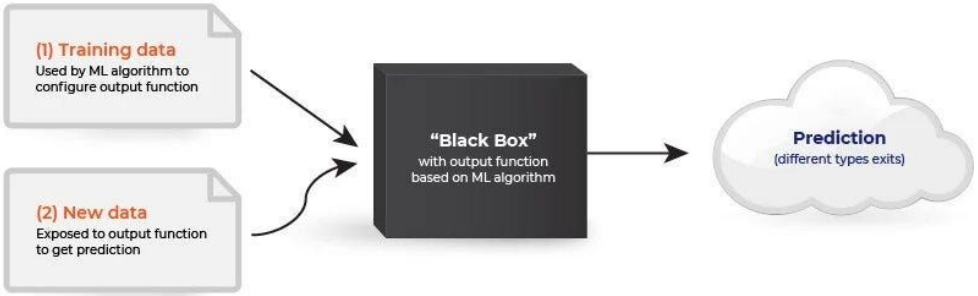
Примеры задач

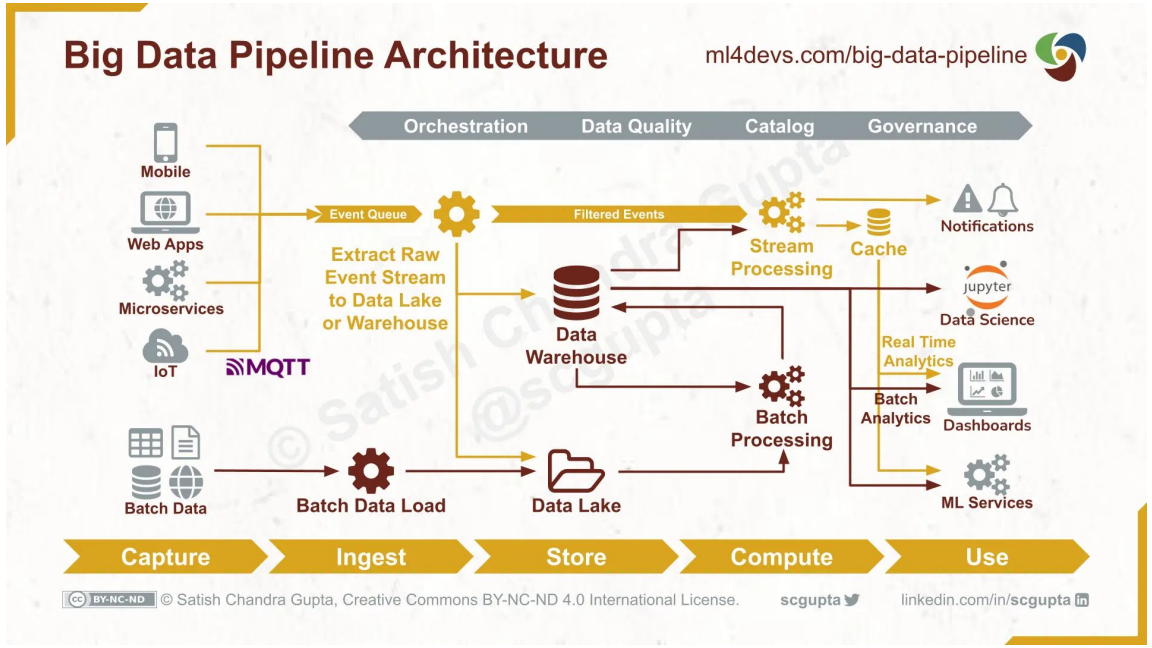
1) Регрессия
   - Предсказание котировок на бирже
   - Предсказание износа шин Белазов
2) Классификация
   - Распознавание кошечек-собачек
   - Определение состояния слива шлака по вибрациям
3) Кластеризация
   - Распознавание аномалий
   - Группировка объявлений
4) Распознавание
   - Автомобильного номера
   - Лиц
   - Оценка шлака и металла в металлургии



Особенности применения ML в проекте

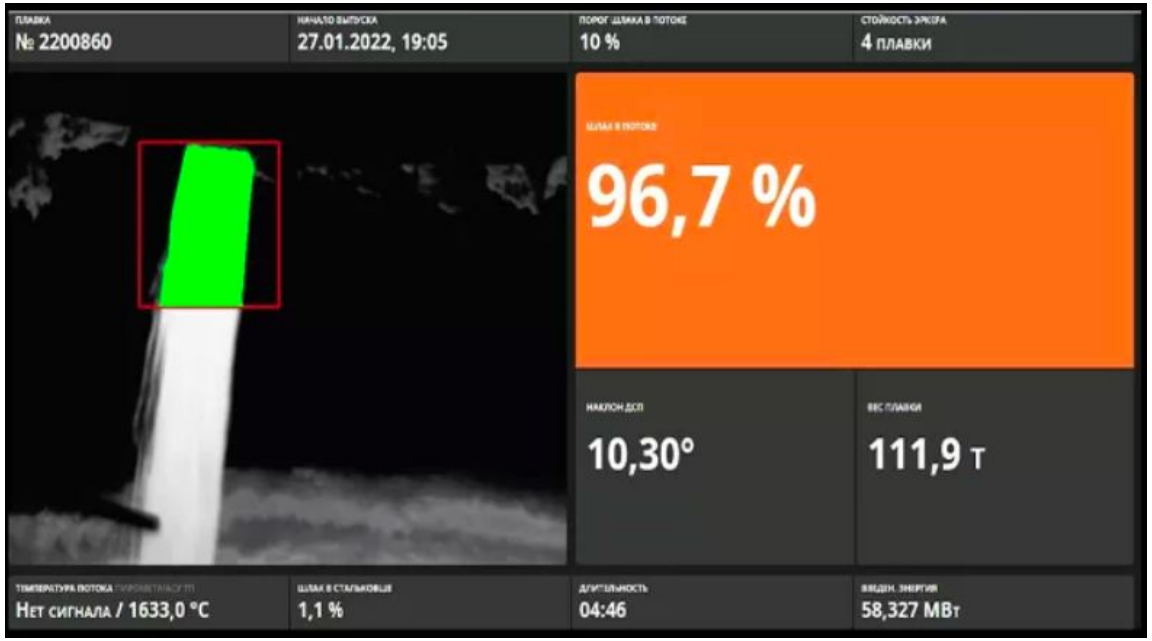
1) Черная ML-коробка для всей ИС - не работает
2) ML - один из элементов ИТ-инфраструктуры
   - Сбор данных
   - Подготовка (преобразование/эмбеддинг, синхронизация, работа с NULL)
   - Хранение
   - API (Взаимодействие с клиентами)
   - Постобработка (восстановление из эмбеддингов)
   - Анализ





Некоторые сценарии

Детектирование шлака при выпуске металла



Простое распознавание изображений (классификация)

ПРОБЛЕМАТИКА

- Попадание шлака в стальковш при выпуске металла
- Высокий остаток металла в конвертере за счет преждевременного перекрытия летки поплавком
- Высокое влияние человеческого фактора

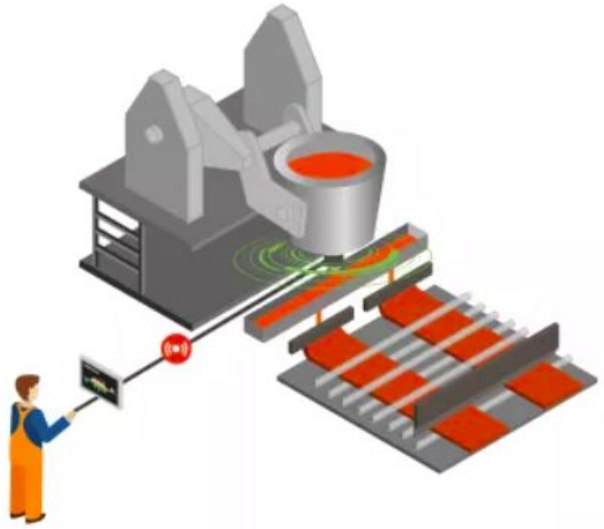
РЕШЕНИЕ

- Формирует рекомендации в режиме реального времени изменить угол наклона конвертера, если соотношение металл/шлак больше или равно пороговому значению
- Оценка наличия шлака в потоке проводится при помощи тепловизионной техники и компьютерного зрения

https://datana.ru/solutions/datana-digital-solutions/detektirovanie-shlaka-pri-vypuske-metalla-iz-konvertera-dsp/

Некоторые сценарии

Отсечка шлака на МНЛЗ



Проблематика

Преждевременное закрытие шибера приводит к избыточным остаткам металла в стальковше

При позднем закрытии шибера происходит попадание шлака в непрерывнолитую заготовку, это приводит к снижению качества металла

Решение

Выдает рекомендацию оператору о необходимости закрыть шиберный затвор «на опережение» с поправкой на время реакции оператора и время срабатывания шиберного механизма

Предусматривается автоматический режим закрытия шибера

Распознавание звука (виброданных) (скользящий буфер)

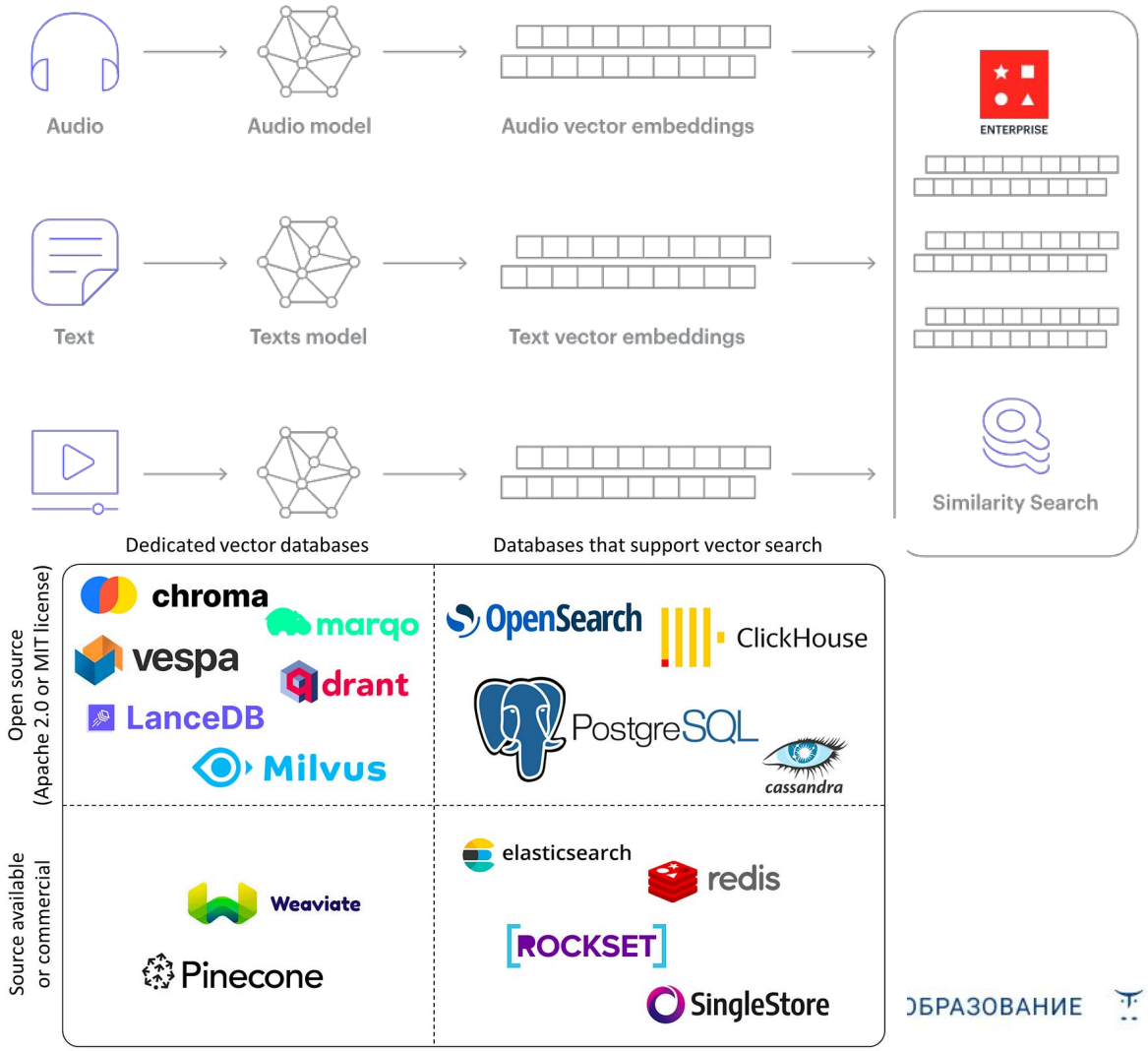
https://datana.ru/solutions/smart/#vibrodetektirovanie-i-otsechka-shlaka

Базы данных для ML

Когда данных очень много

1) Сохранение:
   - С помощью МО-модели вычисляем эмбеддинги
   - Сохраняем их в векторной БД
2) Поиск:
   - С помощью МО-модели вычисляем эмбеддинги
   - Поиск похожих эмбеддингов в векторонй БД

Пример: база данных лиц, отпечатков пальцев и пр.

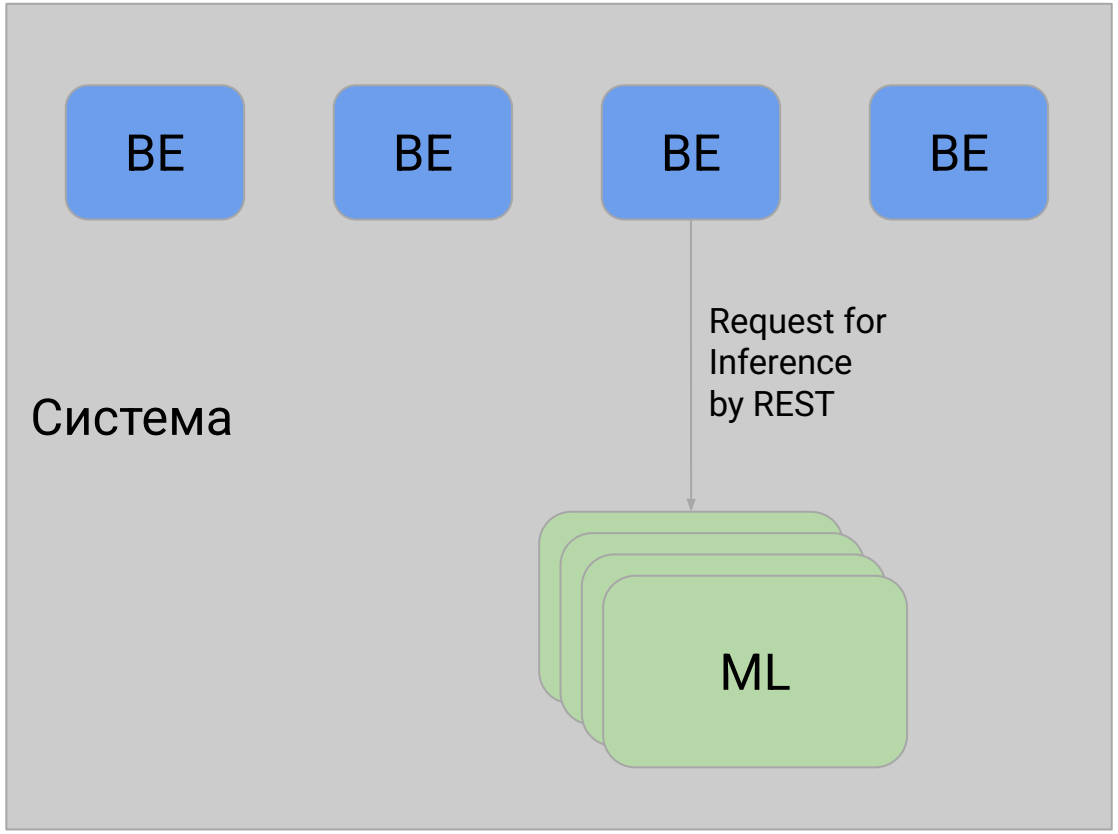


Интеграция ML в BE

Основные проблемы интеграции ML-BE

Идеальная схема интеграции

1) Отдельный микросервис (MS) с ML-моделью
2) Бэкенд взаимодействует с ML-MS по HTTP



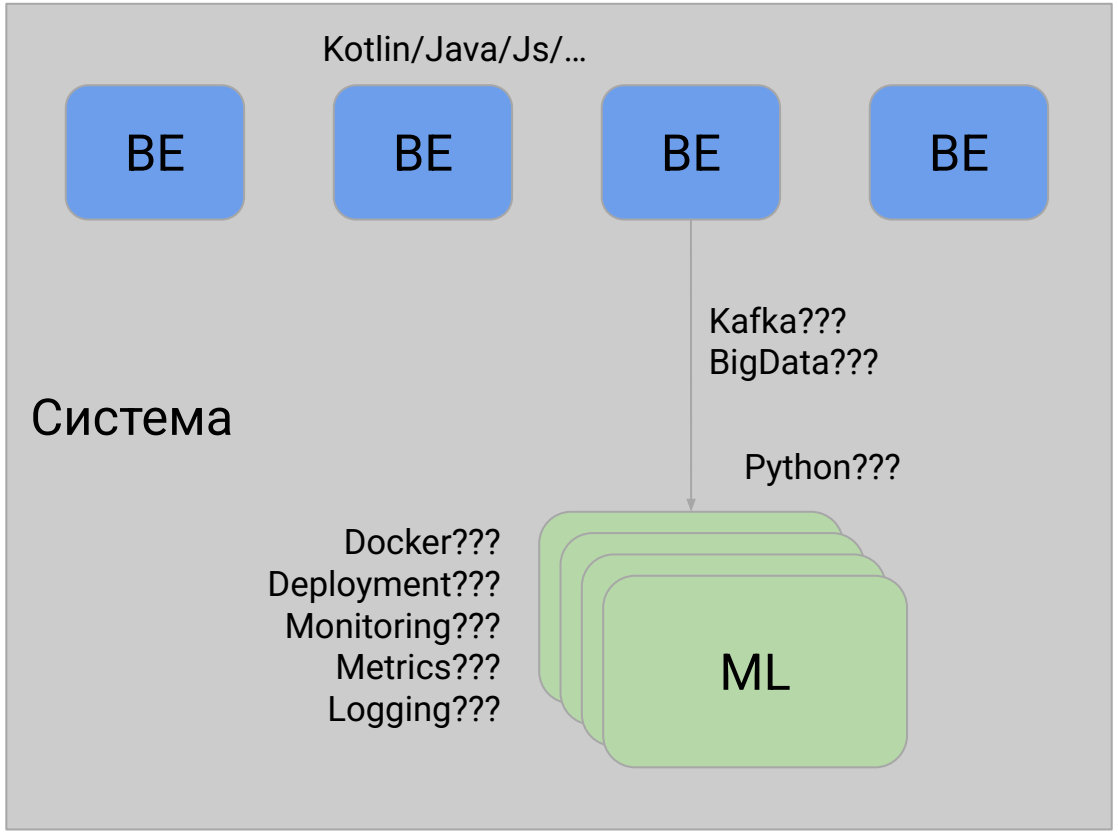
Основные проблемы интеграции ML-BE

Идеальная схема интеграции

1) Отдельный микросервис (MS) с ML-моделью
2) Бэкенд взаимодействует с ML-MS по HTTP

Реалии

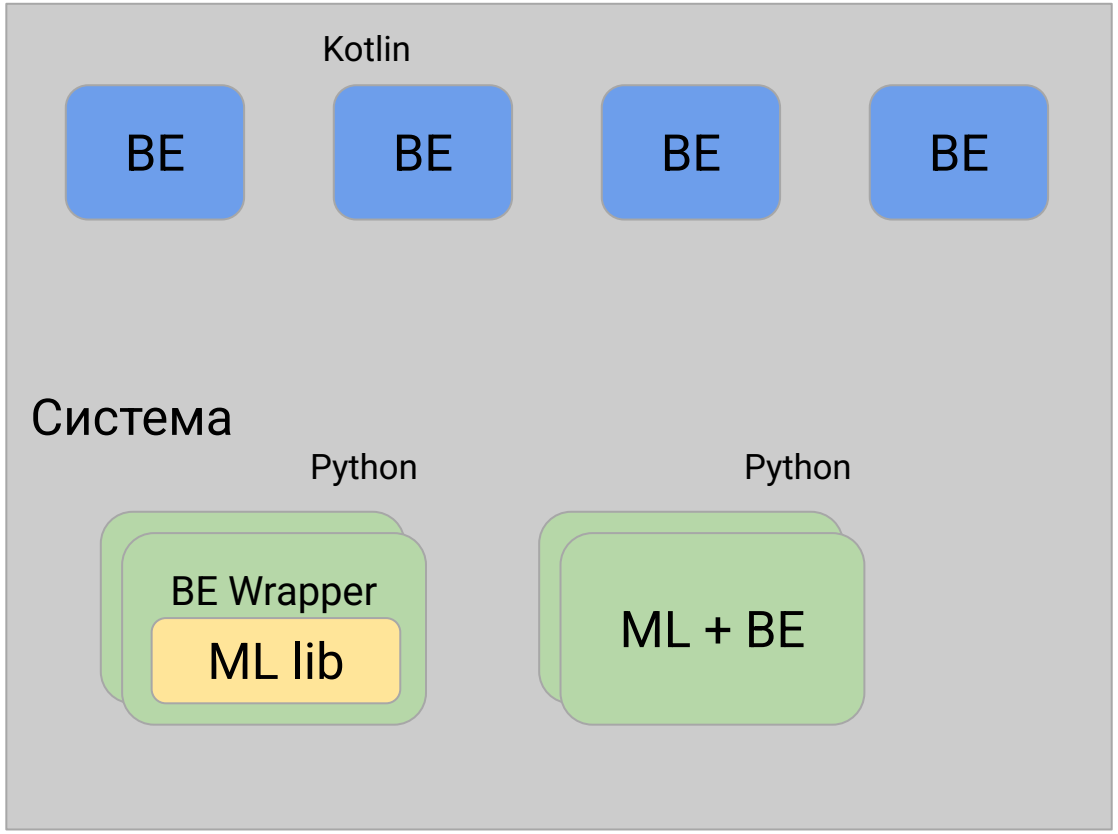
1) Дата-инженеры не обязаны быть специалистами по разработке High-Load (кэш, rps)
2) Не всегда подходит интеграция по HTTP
3) Требуется много дополнительных обвязок: мониторинг, метрики, тестирование, контейнеры, логирование и пр.
4) Другой стек на BackEnd (Java/JS/…) - проблема сертификации



Способы интеграции

На базе Python

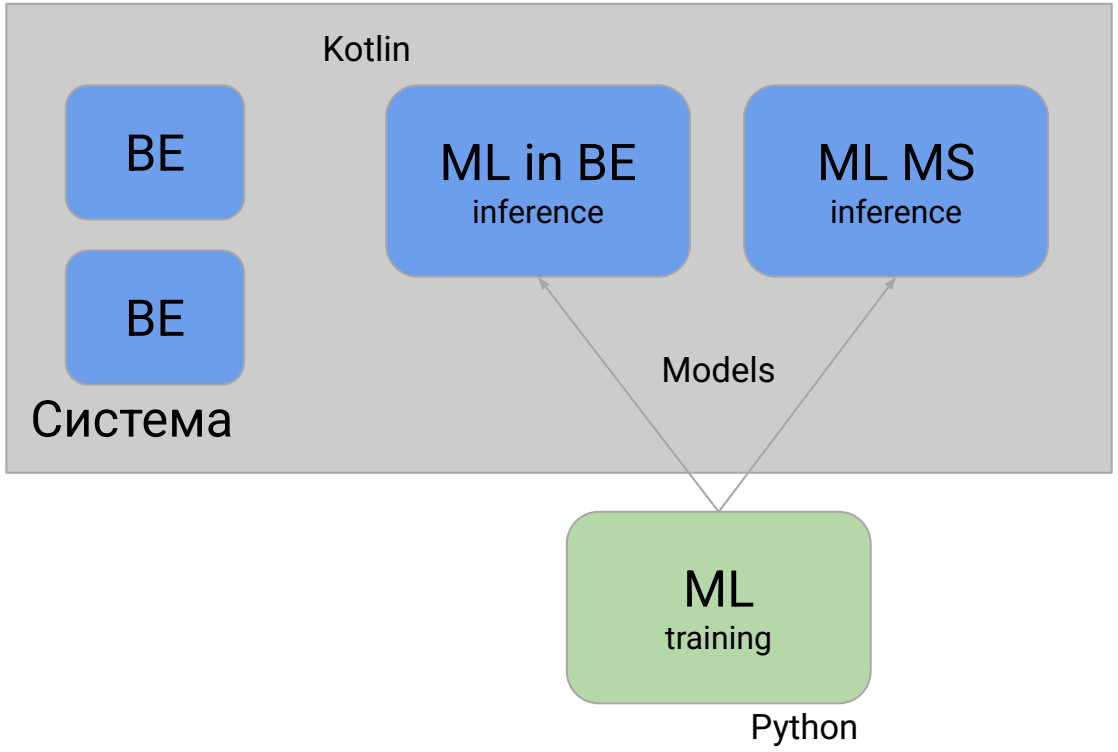
1) Совместная работа команд над одним кодом
2) BE team - приложение, ML team - библиотеку ML



Способы интеграции

На базе Kotlin

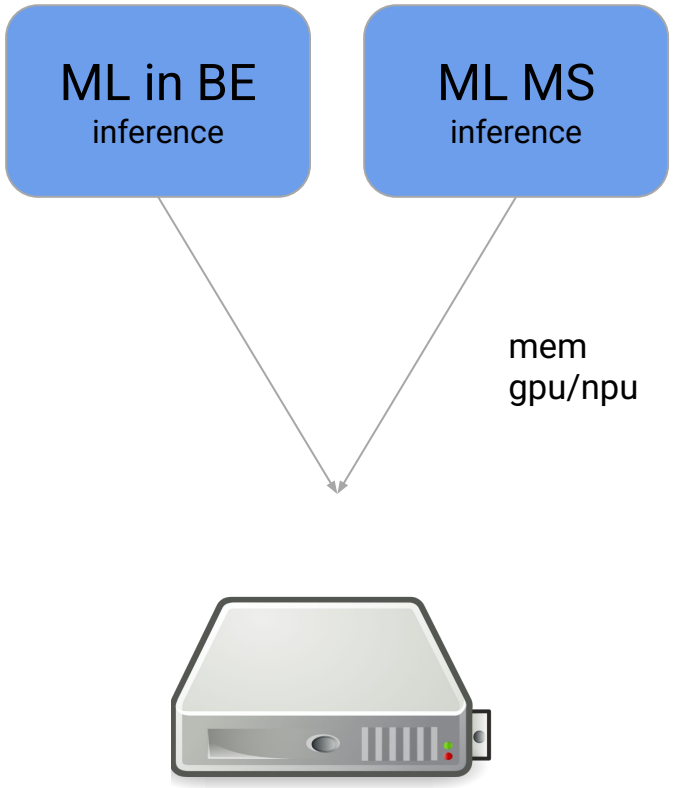
1) Обучение на Python
2) Inference на Kotlin
   a) Отдельный микросервис
   b) Встраивание в бэкенд



Технические вопросы

На базе Kotlin

1) Как интегрироваться
   a) Интероперабельность фреймворков - нет
   b) ONNX
   c) IREE
2) Большие модели
   a) Кластерные решения
   b) Оптимизация моделей
3) Особые требования к оборудованию
   a) память
   b) GPU/NPU
4) Ограниченность поддерживаемых платформ (Kotlin Native не поддерживается)



Оборудование

Оборудование для ML

Главная проблема - большой объем вычислений

1) CPU - Central Processor Unit
2) GPU - Graphic Processor Unit
3) TPU - Tensor Processor Unit
4) NPU - Neuro Processor Unit
5) BPU - Brain Processing Unit, brain processors
6) DPU - Deep learning Processing Unit
7) FPGA - Field Programmable Gate Arrays

На CPU только небольшие задачи

CPU
- Small models
- Small datasets
- Useful for design space exploration

GPU
- Medium-to-large models, datasets
- Image, video processing
- Application on CUDA or OpenCL

TPU
- Matrix computations
- Dense vector processing
- No custom TensorFlow operations

FPGA
- Large datasets, models
- Compute intensive applications
- High performance, high perf./cost ratio

GPU - интерфейсы

1. cuDNN (CUDA Deep Neural Network library)

Библиотека Nvidia для глубокого обучения. Проприетарный и закрытый

2. NVIDIA TensorRT

Оптимизатор и ускоритель инференса нейросетей на GPU при инференсе

3. ROCm (Radeon Open Compute)

Платформа для вычислений на GPU AMD

4. TPU для TensorFlow

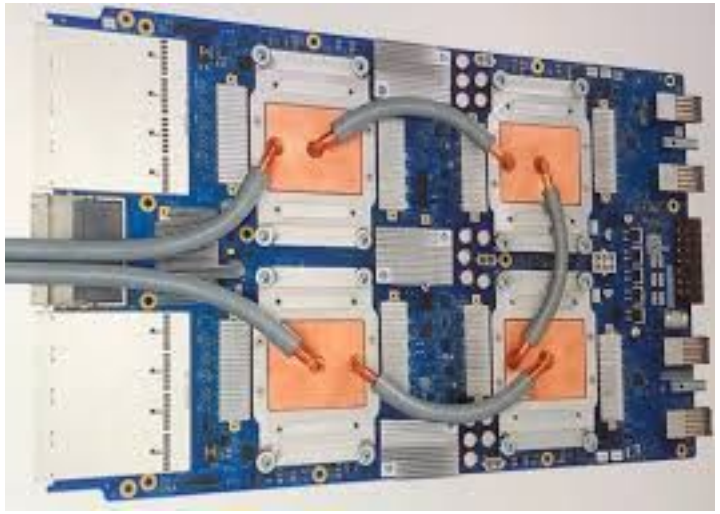
TensorFlow поддерживает Tensor Processing Units от Google

5. Vulkan

Свободный API для GPU

6. OpenCL

Старыйы свободный API для GPU



Тензорный процессор Google

Облака для обучения с мощным GPU

Compute Cloud | Object Storage | Kubernetes® | PostgreSQL | ClickHouse | MongoDB | MySQL®

Redis™ | Elasticsearch | Apache Kafka® | Greenplum® | GitLab® | SpeechKit | Translate | Tracker

SmartCaptcha | Техническая поддержка

Compute Cloud

Платформа	AMD EPYC™ with NVIDIA® Ampere® A100
Количество GPU	1 2 4 8
Количество vCPU	28
Объём RAM	119 ГБ
Прерываемая ВМ	☐
Публичный IP-адрес	☐
Исходящий трафик	0 ГБ
Диск – 1 (Загрузочный)	HDD ◉ SSD
Размер диска	1085 ГБ (5 ГБ – 4096 ГБ)

Добавить диск

Итого:

Compute Cloud × 1	322 680,74 ₽
AMD Epyc with Nvidia A100. 100% vCPU	18 144,00 ₽
AMD Epyc with Nvidia A100. GPU	267 624,00 ₽
AMD Epyc with Nvidia A100. RAM	23 990,40 ₽
Быстрое сетевое хранилище (SSD)	12 922,34 ₽
Исходящий трафик в интернет	0,00 ₽
322 680,74 ₽	в месяц

Перейти в консоль

Поделиться расчётом

Получить консультацию

Интеграция

Интеграция: ONNX

ONNX - open neural networks exchange

ONNX (Open Neural Network Exchange) позволяет преобразовывать модели из различных фреймворков (TensorFlow, PyTorch, Sklearn и др.) в единый формат.

Главный контрибутор - Microsoft

ONNX Runtime для выполнения модели в Kotlin-приложении

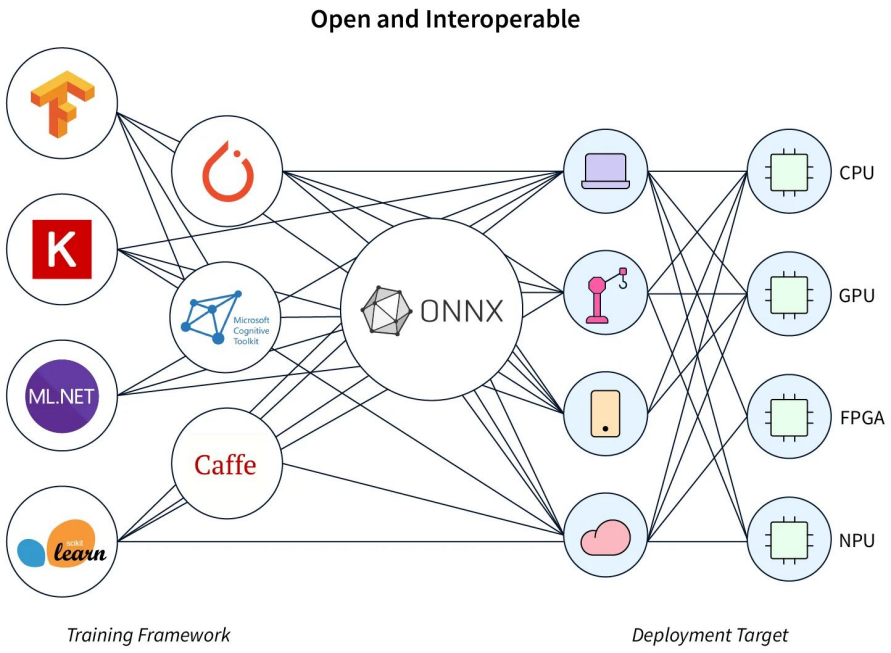
Преимущества:

- Поддержка различных фреймворков.
- Эффективное выполнение модели.

Недостатки:

- Дополнительный шаг преобразования модели в ONNX формат.
- Ограниченная поддержка некоторых специфических операций и функций.

https://onnxruntime.ai/docs/get-started/



SCALER Topics

ONNX - open neural networks exchange

https://onnx.ai/supported-tools.html

https://github.com/onnx/onnxmltools

ONNXMLTools enables you to convert models from different machine learning toolkits into ONNX. Currently the following toolkits are supported:

- Tensorflow (a wrapper of tf2onnx converter)
- scikit-learn (a wrapper of skl2onnx converter)
- Apple Core ML
- Spark ML (experimental)
- LightGBM
- libsvm
- XGBoost
- H2O
- CatBoost

Pytorch has its builtin ONNX exporter check here for details

ONNX Runtime

Оптимизированный инференс на CPU

Инференс на GPU

https://onnxruntime.ai/

Kotlin/Kotlin Multiplatform - нет

Python | C# | JavaScript | Java | C++ | More..

```java
import ai.onnxruntime.*;
// Load the model and create InferenceSession
String modelPath = "path/to/your/onnx/model";
OrtEnvironment env = OrtEnvironment.getEnvironment();
OrtSession session = env.createSession(modelPath);
// Load and preprocess the input image to inputTensor
...
// Run inference
OrtSession.Result outputs = session.run(inputTensor);
System.out.println(outputs.get(0).getTensor().getFloatBuffer().get(0));
```

- Python
- C++
- C
- C#
- Java
- JavaScript
- Objective-C
- Julia and Ruby APIs
- Windows
- Mobile
- On-Device Training
- Large Model Training

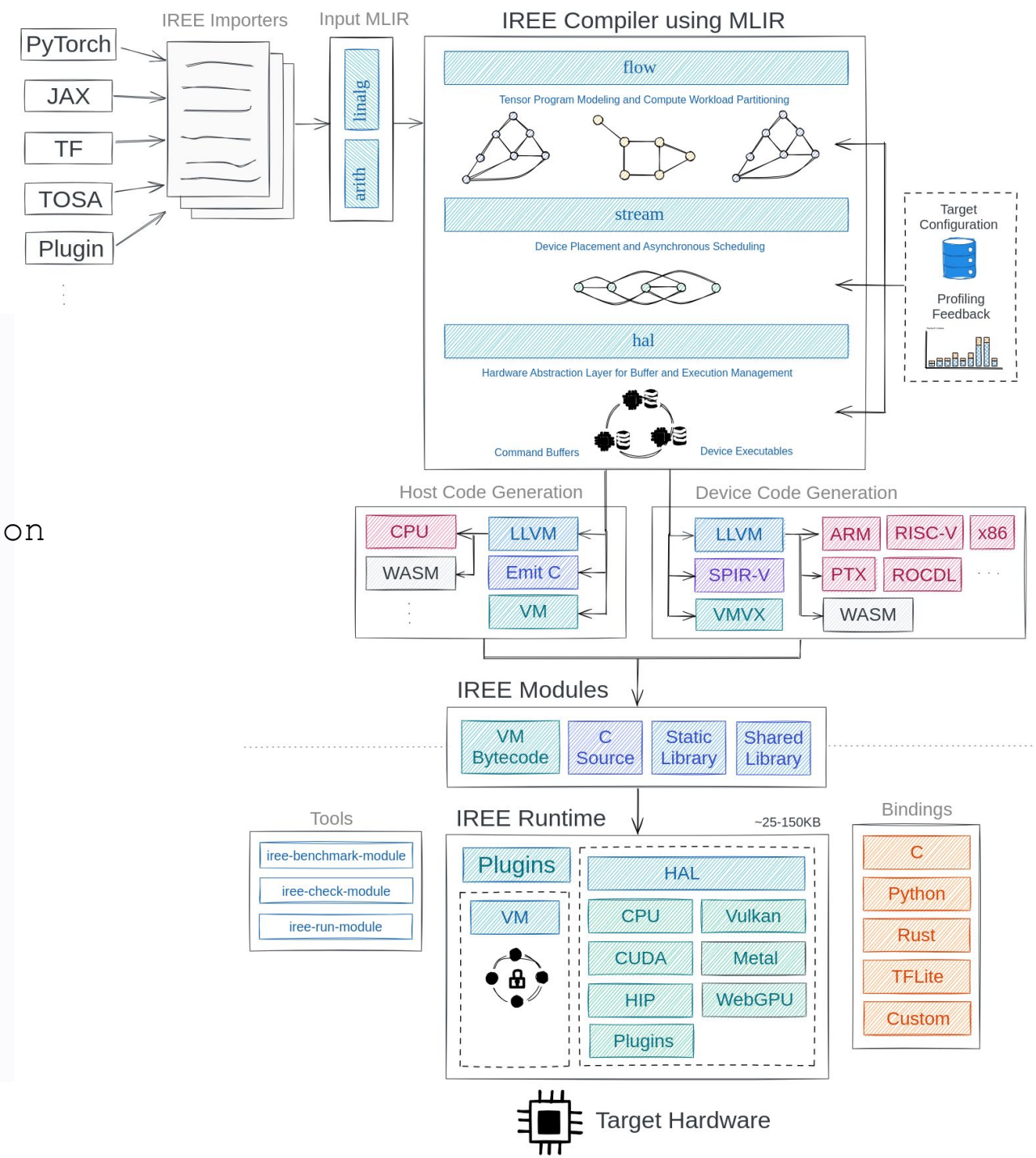
Интеграция: IREE

IREE

Конкурент ONNX, появился позже, более амбициозный

IREE (Intermediate Representation Execution Environment)

Multi-Level Intermediate Representation
https://mlir.llvm.org/



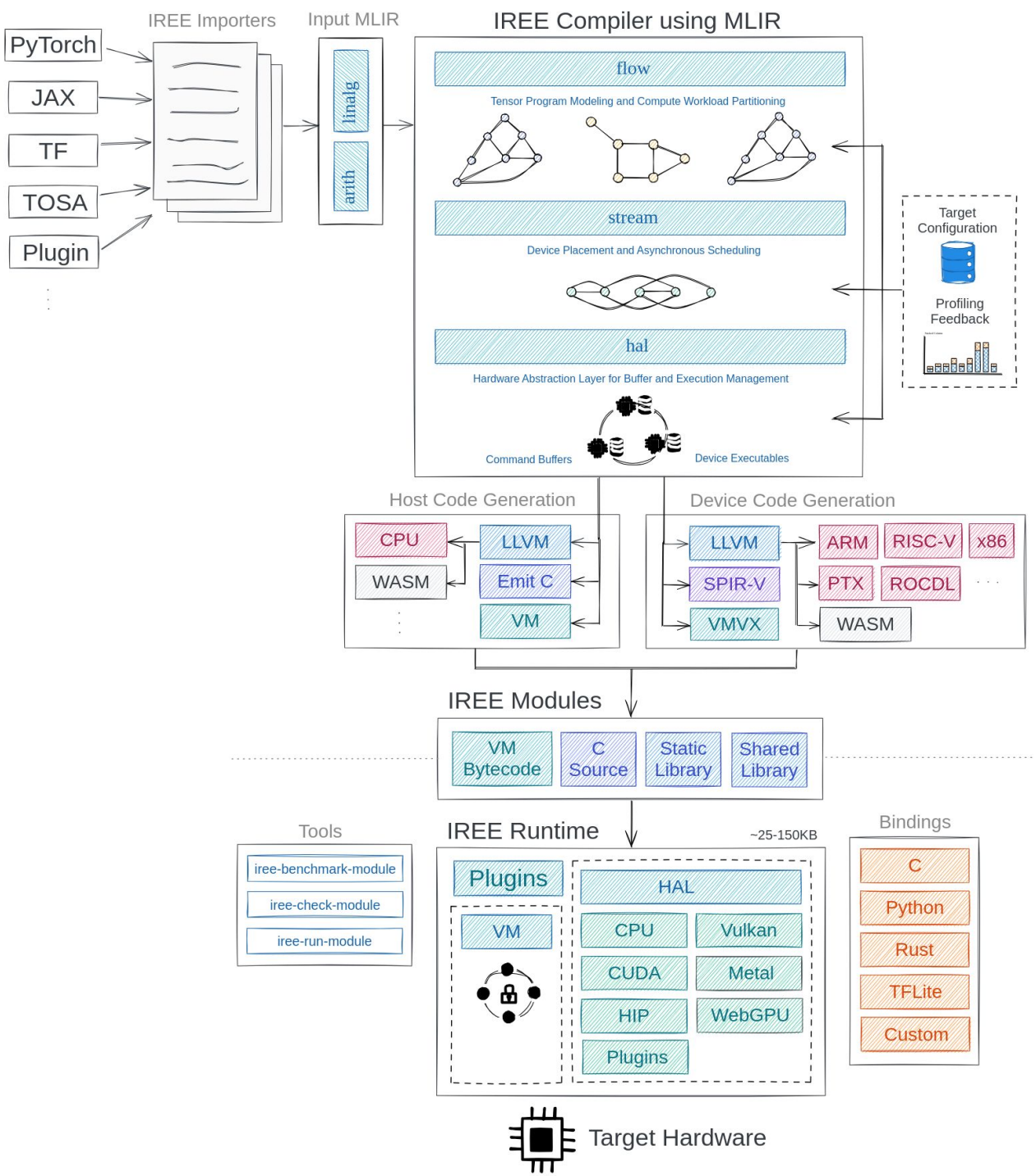
IREE

Особенности:

- Ahead-of-time compilation
- Support for advanced model features
- Designed for CPUs, GPUs, and other accelerators
- Low overhead, pipelined execution
- Binary size as low as 30KB on embedded systems

На ранней стадии

В Kotlin сообществе дискуссии об участии

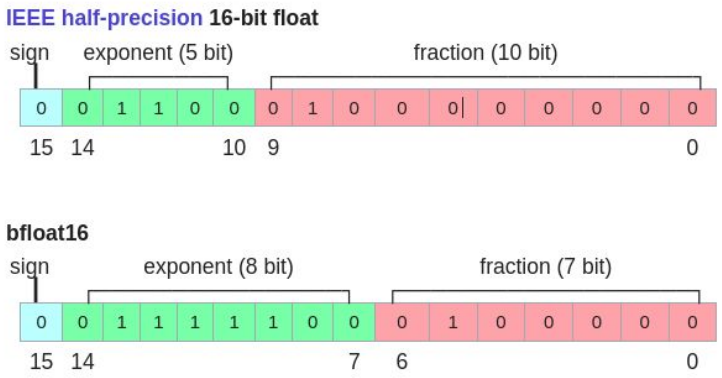


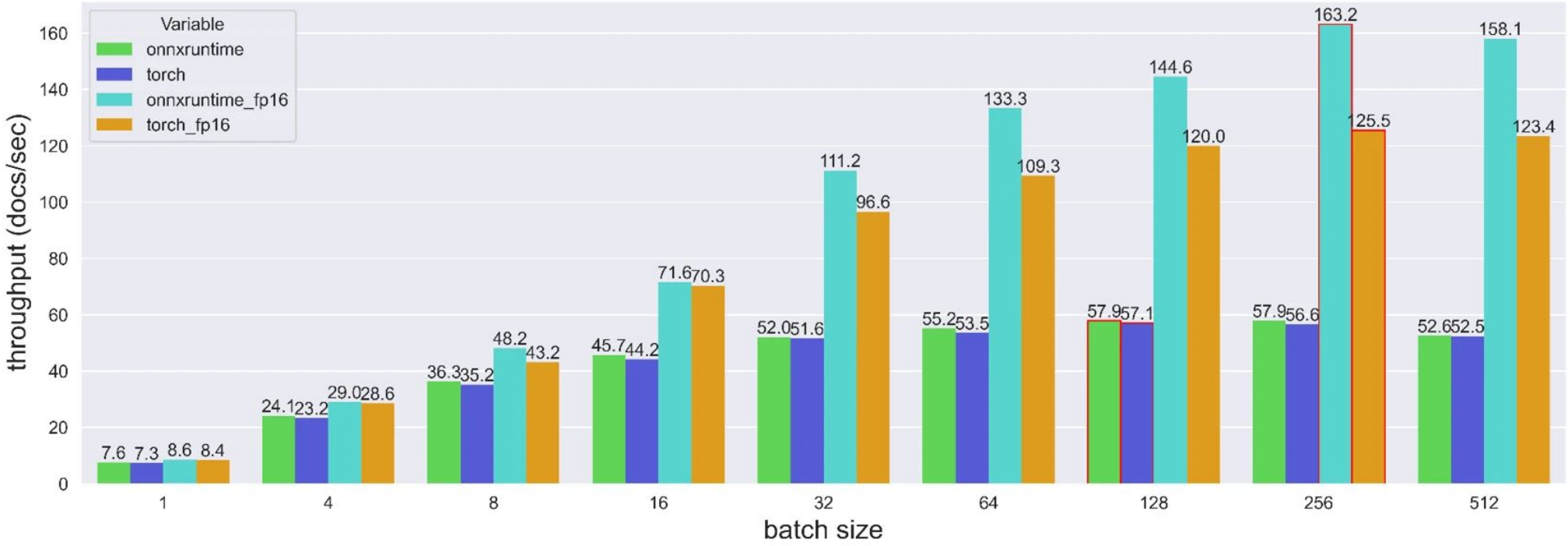
Оптимизация

Квантизация

Понижение точности вычислений модели на инференсе

- модель занимает меньше памяти
- вычисляется быстрее
- float64 -> float32 -> float16 / bfloat16 | (а ещё бывает -> int8 -> int4 -> int3)





МО в Kotlin

Библиотеки для ML (Kotlin)

1. KInference

https://github.com/JetBrains-Research/kinference

JVM, JS. CPU, CUDA

1. KotlinDL

https://github.com/Kotlin/kotlindl

JVM, Android. CPU, GPU Tensorflow

Библиотеки для ML (Java)

1. DeepLearning4J DL4J

https://deeplearning4j.konduit.ai/

CPU, CUDA

1. ONNX

https://github.com/microsoft/onnxruntime-extensions

C, C++, Objective-C, C#, Python, JS, **Java**, Julia, Ruby

DL4J

Deep Learning for Java

Вопросы?

Ставим “+”,
если вопросы есть

Ставим “–”,
если вопросов нет

LIVE

Рефлексия

Слайд с тезисами

Подведем итоги

1. Задачи для МО: регрессия, классификация, кластеризация
2. Большинство задач делаются простыми, в некоторых случаях применяются векторные БД
3. Для МО используется специализированное оборудование: NPU, BPU, DPU, FPGA. В большинстве случаев GPU на CUDA. Есть движение в сторону Vulkan
4. Интеграция в бэкенд с помощью микросервиса по HTTP, безопаснее в виде Python-библиотеки или на языке бэкенда через ONNX
5. Есть библиотеки под Kotlin, но не под Kotlin Native. Наиболее универсальная - ONNX от Microsoft

Рефлексия

С какими основными мыслями и инсайтами уходите с вебинара?

Достигли ли вы цели вебинара?

Цели вебинара

Проверка достижения целей

1. Иметь представление о задачах, решаемых с помощью Искусственного интеллекта и Машинного обучения и способах их решения
2. Знать как МО интегрируется в ИТ-инфраструктуру и как происходит взаимодействие с другими компонентами системы
3. Решать задачи интеграции МО в приложение на Kotlin

Следующий вебинар

31.10.2025

Модуль 8: Проектная работа

Обзор значимых проектов на Kotlin, которые выходят за рамки курса

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/202505-ok-marketplace
4. Vector DB: https://www.datacamp.com/blog/the-top-5-vector-databases
5. ONNX: https://onnx.ai/
6. IREE: https://iree.dev/
7. Оборудование: https://habr.com/ru/companies/recognitor/articles/524980/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары